

ABSTRACTION

- In C#, an abstract class serves as a **blueprint** for other classes, embodying the concept of abstraction.
-
- It defines a **common** structure and behavior that derived classes must adhere to, **without** providing a complete implementation itself.

- **hide** the complex implementation details of a class from the outside world and expose only the necessary functionalities to the user.
- hide non-essential details
- focuses on "what" an object does rather than “how”

- Abstract methods
- Abstract classes

ABSTRACT CLASSES

- An abstract class **CANNOT** be instantiated directly. It serves as a base class for other classes to inherit from.
- can contain both **abstract methods** (without implementation, requiring derived classes to provide them) and **concrete / non-abstract methods** (with implementation).
- Abstract methods are declared using the **abstract** keyword and have no body.
- Derived classes **must** implement all abstract methods inherited from the abstract base class.


```
public abstract class Shape
{
    public abstract double GetArea(); // Abstract method
    public void DisplayMessage() {
        Console.WriteLine("This is a shape."); // Concrete method
    }
}

public class Circle : Shape {
    public double Radius { get; set; }
    public Circle(double radius) {
        Radius = radius;
    }

    public override double GetArea() { // Implementation of abstract method
        return Math.PI * Radius * Radius;
    }
}
```

ABSTRACT CLASSES' CONSTRUCTOR

- In C#, **abstract** classes **CAN** indeed have constructors, despite the fact that you **CANNOT** directly create an instance of an abstract class.

ABSTRACT CLASSES' CONSTRUCTOR

- The primary purpose of these constructors is to
 - **initialize** the **fields** and **properties** defined within the abstract class and. These **fields/ properties** will be present in all derived classes, and the constructor ensures their proper setup
 - enforce **invariants** that all derived classes will inherit.

`base()` is used to call abstract class' constructor

- The constructor of an abstract class is implicitly called when a non-abstract **derived** class that inherits from it is instantiated.
- The derived class's constructor will explicitly or implicitly call the base abstract class's constructor using the `base()` keyword.

PROTECTED

- Abstract class constructors are typically declared as **protected**.
- This ensures that they can only be accessed and called by derived classes, preventing direct instantiation from outside the inheritance hierarchy.
- Public constructors in abstract classes are generally discouraged as they suggest a misunderstanding of abstract class purpose.

```

public abstract class Shape
{
    private string s = "Shape";
    protected Shape(string s) {
        this.s = s;
        Console.WriteLine($"Shape's constructor is called.
                           This is a shape of {s}");
    }
    public abstract double GetArea(); // Abstract method
    public void DisplayMessage() {
        Console.WriteLine( $"This is a shape of {s}."); // Concrete method
    }
}

public class Circle : Shape {
    public double Radius { get; set; }
    public Circle(double radius): base("Circle"){
        Radius = radius;
    }

    public override double GetArea(){// Implementation of abstract method
        return Math.PI * Radius * Radius;
    }
}

```


C# CONSTRUCTOR CALLS

Rules:

- The Base Class Constructor is Always Called First

1. Implicit Call to DEFAULT Base Constructor:

- If the base class has a **default/parameterless** constructor (either explicitly **defined** or **provided** by the compiler) and the derived class constructor does not explicitly call a specific base constructor, the **default /parameterless** base constructor will be implicitly called.

2. Explicit Call using `base()` Keyword:

- To explicitly call a specific constructor of the base class (especially if it requires parameters), the `base` keyword is used within the derived class constructor's initializer list.

NOTE

- If the base class has constructors other than default constructor and the derived class calls a default base class constructor, this will issue an **ERROR**.

```

public abstract class Shape
{
    private string s = "Shape";
    protected Shape() {
        Console.WriteLine("Shape's DEFAULT constructor is called");
    }
    protected Shape(string s) {
        this.s = s;
        Console.WriteLine($"Shape's constructor is called.
                          This is a shape of {s}");
    }
    public abstract double GetArea(); // Abstract method
    public void DisplayMessage(){
        Console.WriteLine( $"This is a shape of {s}."); // Concrete method
    }
}

public class Circle : Shape {
    public double Radius { get; set; }
    public Circle(double radius): base("Circle"){ // try base() and try no explicit call
        Radius = radius;
    }

    public override double GetArea(){// Implementation of abstract method
        return Math.PI * Radius * Radius;
    }
}

```


BENEFITS OF ABSTRACTION

- Allows for **different** implementations of the **same** **abstract** behavior.
- Reduces complexity by **hiding** internal implementation details.
- Easier to maintain code without affecting other parts of the system.